

Rapport Engineering Projects
Inetum App Scanner

Ahmad Houhou, Cédric Alonso, Jade Hatoum, Matthieu Brandao, Guillaume Gomez, Sirine Yahia

FP-25001-20260209

Remerciements

Nous tenons à exprimer notre profonde gratitude aux personnes qui ont contribué à la réussite de ce projet.

Nous remercions tout particulièrement notre menteur Julien Saïd pour sa disponibilité constante tout au long de ce projet. Ses conseils, son accompagnement bienveillant et son écoute attentive nous ont permis de progresser et de surmonter les difficultés rencontrées.

Nous adressons également nos sincères remerciements à Stephen Perin pour la confiance qu'il nous a accordée en nous confiant ce projet. Nous le remercions également pour le temps qu'il nous a consacré et pour ses retours constructifs qui ont enrichi notre réflexion.

Enfin, nous remercions l'ensemble des personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet.

IAS, En quelques lignes

Résumé - Inetum App Scanner

De nombreuses organisations s'appuient encore sur des systèmes legacy de plusieurs millions de lignes de code, difficiles à maintenir et à sécuriser. Les solutions d'analyse existantes sont souvent coûteuses, complexes à déployer et à utiliser.

Développé dans le cadre d'un projet de fin d'études en partenariat avec Inetum, Inetum App Scanner (IAS) est un outil d'analyse et de visualisation d'architectures applicatives, automatisé par intelligence artificielle générative. Il permet de cartographier les dépendances d'un projet, de détecter les vulnérabilités et de produire des rapports d'audit exploitables — le tout en exécution locale, sans exfiltration du code source.

Ce POC de niveau TRL 4 vise à démontrer qu'il est possible de concilier profondeur d'analyse, facilité d'utilisation et maîtrise totale des données, à un coût significativement inférieur aux solutions propriétaires du marché.

Résultats du POC

Pipeline opérationnel en 8 étapes :

1. Download
2. Cleanup
3. Dependency Scan
4. Vector Index
5. AI Source Code Analysis
6. Audit Reporting
7. HTML Report
8. Upload

Double détection des vulnérabilités : OWASP Dependency-Check (CVE/CVSS) + analyse IA par Mistral avec scanner heuristique de secours

Architecture modulaire et scalable :

- .NET 8 (API & Worker)
- React/TypeScript (Frontend)
- RabbitMQ (orchestration)
- Neo4j (graphe)
- ChromaDB (embeddings)
- MongoDB (état des jobs)
- Azurite (stockage)

Traitement automatisé avec politique de retry (3 tentatives, Dead Letter Queue) et gestion d'erreurs robuste

Rapports d'audit JSON et HTML générés automatiquement

Stack Technologique

Stack 100 % open-source : Ollama/Mistral (analyse IA et embeddings via nomic-embed-text), Neo4j 5.26 (graphe de dépendances et vulnérabilités), ChromaDB 0.4 (mémoire vectorielle du code), RabbitMQ 3 (orchestration asynchrone avec retry/DLQ), MongoDB 7 (persistance des jobs), OWASP Dependency-Check (scan CVE/CVSS via NVD). Architecture pipeline modulaire permettant l'ajout facile de nouveaux langages et types d'analyse.

Conclusion

Le POC démontre la faisabilité technique d'une alternative crédible et économique aux solutions qui dominent actuellement le marché.

Mots clés

Nom	Description
Modèles d'Intelligence Artificielle	Système mathématique entraîné sur des données ciblées pour identifier, prédire et générer des résultats de façon autonome.
Sécurité	Ensemble des mesures visant à protéger l'intégrité et la confidentialité des données d'un système.
Vulnérabilités	Failles de sécurité (CVE) présentes dans le code ou les dépendances.
Code Legacy	Code ancien, souvent mal documenté ou utilisant des technologies obsolètes.
Audit	Processus d'examen pour vérifier la conformité des licences et l'absence de failles connues.

Historique de version

Nom	Date	Changement
Equipe IAS	01/02/2026	Initialisation du rapport
Equipe IAS	08/02/2026	Edition du rapport
Equipe IAS	09/02/2026	Finalisation du rapport

Table des matières

Remerciements **2**
IAS, En quelques lignes **3**
Mots clés **4**
Historique de version **4**
Table des matières **5**
1. Introduction **7**
1.1 Présentation du Partenaire 7
1.2 Problématique et Besoin Complexe 7
1.3 Objectifs du Projet ING3 7
2. Documents et terminologie **8**
2.1 Glossaire 8
2.2.1 Acronymes 8
Description du Projet **8**
3.1 Introduction et Enjeux Stratégiques 8
3.2 Besoins du Marché et État de l'Art 8
3.3 Description du Produit 9
3.4 Caractère Innovant de la Solution 11
3.5 Liberté d'Exploitation et Propriété Intellectuelle 12
3.6 Contraintes Réglementaires 13
3.7 Positionnement Marché et Avantage Concurrentiel 13
3.8 Impacts Environnementaux et Éco-Conception 14
3.9 Programme de Travail et Jalons 14
5. Fonction et contribution des acteurs **15**
6. Études de faisabilité technique réalisées **16**
6.1 Environnement physique 16
6.2 Environnement matériel 16
6.3 Environnement logiciel 16
6.4 Données 18
Données d'entrée (projets à analyser) 18
Données de référence 18
6.5 Analyse de marché et positionnement concurrentiel 18
6.6 Évaluation des risques techniques 19
6.7 Approche éco-conception 19
6.8 Scalabilité et contraintes opérationnelles 19
7. Suite de tests **20**
7.1 Exigence 1 : Ingestion des données 20
7.2 Exigence 2 : Pipeline d'analyse complète 20
7.3 Exigence 3 : Détection des vulnérabilités 21
7.4 Exigence 4 : Génération de rapports 22
7.5 Exigence 5 : Résilience et gestion des erreurs 22
7.6 Exigence 6 : Performance de l'analyse 23

7.7 Exigence 7 : Utilisabilité..... 23
7.8 Validation par les parties prenantes..... 24
7.8.1 Validation fonctionnelle..... 24
7.8.2 Validation de performance..... 24
7.8.3 Validation utilisateurs et retours externes..... 24
8. Résultats, Analyse et Interprétation..... 24
8.1 Protocoles de collecte des données..... 24
8.2 Présentation des résultats du POC..... 25
8.2.1 Ingestion et structuration..... 25
8.2.2 Détection des vulnérabilités..... 25
8.2.3 Génération des rapports..... 25
8.2.4 Résilience du pipeline..... 25
8.3 Méthodes d'analyse..... 25
8.3.1 Protocoles de validation des résultats..... 25
8.3.2 Rigueur du cycle de développement (QA & DevOps)..... 26
8.4 Interprétation et discussion..... 26
9. Conclusions et Feuille de route..... 26
9.1 Conclusions..... 26
9.2 Projection et feuille de route..... 27
Annexe..... 27
Use Case : Utilisation fonctionnelle d'Inetum App Scanner..... 27
UC-01 – Soumission d'un projet..... 27
UC-02 – Mise en file d'attente du job..... 28
UC-03 – Démarrage automatique du traitement..... 29
UC-04 – Suivi de l'exécution..... 29
UC-05 – Gestion des erreurs..... 30
UC-06 – Consultation des résultats..... 30

1. Introduction

1.1 Présentation du Partenaire

Inetum est une Entreprise de Services du Numérique (ESN) française fondée en 1970. Elle emploie aujourd'hui environ 28 000 collaborateurs répartis dans une vingtaine de pays pour un chiffre d'affaires d'environ 2,5 milliards d'euros. Le groupe accompagne entreprises et institutions dans leur transformation numérique, avec des expertises couvrant notamment le cloud, la data, l'intelligence artificielle et la cybersécurité.

Au sein d'Inetum, le département dirigé par Stephen Perin intervient en tant que prestataire de services logiciels auprès de grands comptes. Ses équipes assurent le développement, la maintenance et l'évolution d'applications, ce qui les amène à travailler sur de volumineux dépôts de code, dont certains très anciens. Sur ces projets, la documentation s'est progressivement érodée et la dette technique accumulée, freinant la productivité des développeurs et exposant les applications à des risques de sécurité.

1.2 Problématique et Besoin Complexe

Les applications dites « Legacy », c'est-à-dire des systèmes historiques toujours en production mais conçus avec des technologies anciennes, représentent un défi récurrent dans l'industrie du logiciel. Pour les grands comptes clients d'Inetum, qui opèrent dans des secteurs sensibles (défense, institutions publiques, finance), la visibilité sur la dette technique et les vulnérabilités de ces parcs applicatifs est un enjeu critique.

Le projet Inetum App Scanner (IAS) répond à ce besoin en conciliant analyse profonde de bases de code de plusieurs millions de lignes, rapidité d'exécution et souveraineté totale des données. Les solutions du marché s'avèrent souvent inadaptées : trop onéreuses, ou reposant sur des architectures cloud qui imposent l'exfiltration du code source vers des serveurs tiers — inacceptable dans un contexte de cybersécurité. IAS se positionne donc comme une alternative locale et automatisée, pensée pour les environnements sensibles.

1.3 Objectifs du Projet ING3

Ce projet de fin d’étude, mené en équipe dans le cadre de notre formation d'ingénieurs, poursuit plusieurs objectifs :

- **Techniques** : concevoir une architecture cohérente avec le besoin, et intégrer l'IA générative (LLM) dans un processus d'audit à industrialiser.
- **Méthodologiques** : s'organiser en équipe projet en adoptant les pratiques d'un environnement professionnel.
- **Validation** : aboutir à un POC fonctionnel de niveau TRL 4, démontrant la faisabilité de la solution en conditions réalistes.

2. Documents et terminologie

2.1 Glossaire

2.2.1 Acronymes

Acronyme	Définition	Description
RAG	Retrieval-Augmented Generation	Technique consistant à donner au LLM des données externes (code source) pour améliorer ses réponses.
CVE	Common Vulnerabilities and Exposures	Liste publique de failles de sécurité informatique documentées.
TRL	Technology Readiness Level	Échelle d'évaluation du niveau de maturité d'une technologie.

Description du Projet

3.1 Introduction et Enjeux Stratégiques

Les applications dites « Legacy » représentent un défi récurrent dans l'industrie du logiciel. Pour beaucoup de grandes entreprises qui opèrent dans des secteurs sensibles (défense, institutions publiques, finance), chaque modification est un risque de régression, la dette technique continue de s'accumuler tout comme le coût de maintenance et de modernisation.

Le problème est simple : pour moderniser il faut d'abord comprendre. Il faut comprendre l'architecture puis identifier les dépendances et enfin repérer les vulnérabilités enfouies dans des chaînes de composants que plus personne ne maîtrise. Aujourd'hui ce travail d'audit repose sur des outils comme CAST qui sont chers et longs à prendre en main. De plus, ils produisent des résultats dans des formats propriétaires difficilement exploitables.

C'est dans ce contexte que Inetum App Scanner a été conçu. L'idée est de tirer parti de l'IA générative pour automatiser cette phase d'analyse et la rendre accessible ainsi que rapide et économique. Les risques techniques existent bien entendu. La fenêtre de contexte des LLM impose de structurer le code avant de le soumettre. L'interprétation de l'IA n'est pas infaillible ce qui justifie la mise en place d'un scanner heuristique de secours. Et la performance sur de gros projets nécessite une architecture distribuée. Cependant ces risques sont identifiés et gérés dans notre conception.

Le marché visé est large. Il concerne d'abord les grandes entreprises qui possèdent des systèmes legacy massifs. Il s'adresse aussi aux ESN qui réalisent des audits pour leurs clients ainsi qu'aux équipes DevSecOps qui veulent intégrer la sécurité dans leur pipeline.

3.2 Besoins du Marché et État de l'Art

Deux solutions dominent actuellement le marché et chacune présente ses limites.

CAST Imaging / CAST Highlight est le leader historique de la cartographie d'architecture et de l'analyse de dette technique. Le problème c'est le prix qui s'élève à environ 30 000 € pour 25 applications. La courbe d'apprentissage est longue et il faut souvent faire intervenir les experts de l'éditeur pour exploiter les résultats. Par ailleurs, les diagrammes générés sont dans des formats propriétaires ce qui limite l'interopérabilité. Les visualisations souffrent aussi fréquemment de surcharge visuelle ce qui rend l'exploitation compliquée au quotidien.

SonarQube est performant pour ce qu'il fait c'est-à-dire détecter les codes smells, les violations de conventions, les duplications ainsi que les failles de sécurité générales. Cependant il travaille à un niveau de granularité trop fin. Il ne fait pas d'analyse architecturale globale et ne propose pas non plus de stratégie de modernisation.

3.3 Description du Produit

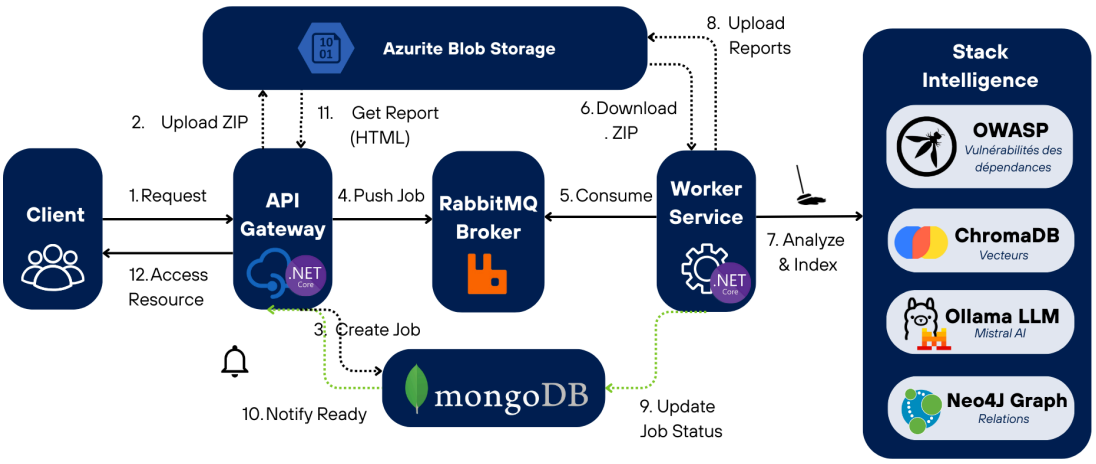
Inetum App Scanner est une solution d'audit de sécurité entièrement automatisée, s'appuyant sur des mécanismes d'intelligence artificielle afin d'identifier les vulnérabilités, les écarts de conformité et les axes d'amélioration des architectures applicatives.

Le processus débute par la soumission d'un projet par l'utilisateur via une API dédiée. Cette soumission peut prendre la forme d'une archive ZIP contenant le code source ou d'une URL pointant vers un dépôt Git distant. Une fois réceptionné, le projet est téléversé dans un stockage **Azure Blob Storage**, garantissant la persistance, la traçabilité et la disponibilité des données tout au long du cycle d'analyse.

À l'issue de cette phase d'ingestion, un **job d'analyse** est généré. Celui-ci encapsule les métadonnées du projet, les paramètres d'exécution ainsi que les règles d'analyse applicables. Le job est ensuite publié dans une **file d'attente RabbitMQ**, assurant un découplage efficace entre la phase de soumission et les traitements d'analyse, tout en permettant une gestion robuste de la montée en charge.

Les **workers**, composants chargés de l'exécution des traitements, surveillent la file d'attente. Dès qu'un worker devient disponible, il récupère le job et initie la **pipeline de traitement**. Cette pipeline orchestre l'ensemble des étapes d'analyse, incluant notamment l'analyse du code source, l'identification des vulnérabilités potentielles et la génération de résultats exploitables.

L'architecture asynchrone et distribuée de la solution permet ainsi d'assurer des performances optimales, une haute scalabilité et une résilience accrue, tout en garantissant la fiabilité et la reproductibilité des analyses réalisées.



L'architecture de la solution repose sur une **séparation claire en trois couches fonctionnelles**, garantissant modularité, scalabilité et robustesse opérationnelle.

La **couche d'exposition** est assurée par une API REST développée en **.NET 8 / ASP.NET Core**, qui constitue le point d'entrée unique du système. Elle prend en charge l'ingestion des projets soumis par les utilisateurs, le déclenchement des analyses, le suivi du cycle de vie des jobs ainsi que la mise à disposition des rapports générés. Cette API intègre par ailleurs des mécanismes transverses essentiels, tels que le *rate limiting*, la validation stricte des entrées et des *health checks*, afin d'assurer la sécurité, la fiabilité et l'observabilité du service.

La **couche de traitement** repose sur un **Worker** également implémenté en **.NET 8**, exécuté sous forme de service de fond. Ce composant consomme les messages depuis **RabbitMQ** et exécute la pipeline d'analyse de manière entièrement asynchrone. Le Worker est configurable via des fichiers de configuration et embarque une gestion robuste des erreurs, incluant une politique de *retry* avec trois tentatives avant la redirection des messages en échec vers une **Dead Letter Queue**, garantissant ainsi la résilience et la traçabilité des traitements.

La **couche de présentation** est assurée par une application **Frontend développée en React 19 avec TypeScript et Ant Design**. Elle permet aux utilisateurs de téléverser leurs projets, de suivre en temps réel l'avancement des analyses et de consulter les rapports générés de manière ergonomique et intuitive.

La **double détection des vulnérabilités** constitue un élément central de la conception du système. D'une part, **OWASP Dependency-Check** est utilisé pour analyser les dépendances applicatives et identifier les vulnérabilités connues (CVE), enrichies de leurs scores de sévérité **CVSS v2 et v3**. D'autre part, un moteur d'analyse basé sur le modèle **Mistral** inspecte directement le code source afin de détecter des failles structurelles qui échappent aux scans de dépendances, telles que les injections SQL, les vulnérabilités XSS ou la présence de secrets codés en dur.

Afin de garantir une couverture minimale même en cas de défaillance de l'analyse par intelligence artificielle (par exemple lorsque la réponse générée n'est pas exploitable), un **mécanisme de secours basé sur des heuristiques et des expressions régulières** est automatiquement activé. Bien que cette approche ne vise pas l'exhaustivité, elle est conçue pour éviter toute omission silencieuse et renforcer la fiabilité globale du dispositif d'audit.

Concrètement voici les différentes étapes de la dite *pipeline* de traitement du worker:

Étape	Description détaillée
1. Download Input	Le code source du projet est récupéré depuis le stockage Blob (Azurite). L'archive est ensuite extraite dans l'environnement de travail du *worker*, préparant ainsi les artefacts nécessaires aux étapes d'analyse ultérieures.
2. Cleanup	Le graphe de données Neo4j est entièrement réinitialisé afin de garantir une analyse sur une base saine. Cette étape évite toute contamination des résultats par des données résiduelles issues d'analyses précédentes.
3. Dependency Scan	L'outil **OWASP Dependency-Check** est exécuté au sein d'un conteneur Docker. Il analyse les dépendances du projet afin d'identifier les vulnérabilités connues (CVE), en s'appuyant sur les bases de données de sécurité publiques, et calcule les scores de sévérité associés (CVSS).
4. Vector Index	Le code source est indexé dans **ChromaDB** à l'aide d'*embeddings* générés par le modèle *nomic-embed-text*, permettant des recherches sémantiques avancées. En parallèle, un graphe structurel représentant les relations **Fichier** → **Classe** → **Méthode** est construit et persisté dans Neo4j.
5. AI Analysis	Un moteur d'analyse basé sur le modèle **Mistral** inspecte chaque fichier du projet afin de détecter des vulnérabilités applicatives telles que les injections SQL, les failles XSS ou la présence de secrets codés en dur. En cas d'échec ou d'incertitude de l'analyse par l'IA, un mécanisme de secours basé sur des règles heuristiques et des expressions régulières est automatiquement déclenché.
6. Audit Reporting	Les vulnérabilités identifiées sont consolidées à partir du graphe Neo4j, puis enrichies avec des extraits pertinents du code source. Un rapport d'audit structuré au format JSON est alors généré, facilitant l'exploitation automatisée des résultats.
7. HTML Report	Un rapport de restitution lisible est produit au format HTML à l'aide du moteur de templates **Scriban**. Ce rapport agrège les résultats de l'analyse des dépendances et de l'audit du code applicatif, offrant une vision synthétique et exploitable des risques identifiés.
8. Upload Output	L'ensemble des résultats (rapports JSON et HTML) est compressé puis téléversé vers le stockage Blob. Ces livrables sont ensuite mis à disposition pour consultation et téléchargement via l'API.

3.4 Caractère Innovant de la Solution

Ce qui distingue IAS des solutions existantes ce n'est pas un seul élément mais plutôt la combinaison de plusieurs approches qui prises ensemble n'existent pas sur le marché.

Le pipeline IA est hybride. Il combine un scan de dépendances classique via OWASP avec une analyse sémantique par LLM via Mistral/Ollama ainsi qu'un fallback heuristique regex. Cette approche en trois couches maximise la couverture de détection tout en gérant le fait que les modèles génératifs ne sont pas toujours fiables.

Le code est stocké sous forme de mémoire vectorielle. ChromaDB conserve les embeddings du code source via nomic-embed-text ce qui permet une recherche sémantique ainsi qu'un enrichissement contextuel des analyses. Cette approche de type RAG (Retrieval-Augmented Generation) est nouvelle dans le domaine de l'audit applicatif.

Les résultats sont modélisés en graphe. Neo4j stocke les relations File → Class → Method → Vulnerability et offre ainsi une cartographie navigable des dépendances et des failles. C'est très différent des rapports PDF statiques que produisent les outils classiques.

L'architecture est event-driven et résiliente. RabbitMQ avec sa politique de retry et sa Dead Letter Queue garantissent que le pipeline ne plante pas silencieusement. De plus, chaque étape est indépendante et remplaçable ce qui permet d'intégrer de nouveaux analyseurs sans tout réécrire.

La stack est 100 % open-source. Il n'y a aucune licence propriétaire ce qui assure une souveraineté totale sur les données et les traitements. L'IA tourne en local via Ollama donc aucune donnée ne quitte l'infrastructure du client. C'est un argument fort face à CAST qui impose ses formats fermés et son cloud.

Le résultat concret est un coût réduit de 60 à 80 % par rapport à CAST pour une couverture d'analyse plus large grâce à l'IA.

3.5 Liberté d'Exploitation et Propriété Intellectuelle

Toute la stack d'IAS repose sur des composants open-source sous licences permissives. Le tableau ci-dessous résume les implications pour chaque brique :

Composant	Licence	Implications
.NET 8 / ASP.NET Core	MIT	Usage commercial libre et pas de copyleft
Neo4j Community	GPL v3	Utilisable en interne mais attention si distribution
RabbitMQ	MPL 2.0	Usage commercial libre avec obligations mineures
ChromaDB	Apache 2.0	Usage commercial libre
Mistral 7B	Apache 2.0	Exécution locale donc aucune fuite de données
OWASP Dependency-Check	Apache 2.0	Outil de référence en sécurité
React / Ant Design	MIT	Usage commercial libre

Le risque de contrefaçon est faible. IAS n'utilise aucune technologie brevetée identifiée. Les algorithmes d'analyse ainsi que les prompts IA et les heuristiques regex et le pipeline d'orchestration sont des développements originaux de l'équipe. La base MITRE (CVE/CWE) est un référentiel public librement accessible. À notre connaissance aucun brevet ne couvre spécifiquement l'approche combinant LLM local avec graphe de dépendances et mémoire vectorielle pour l'audit applicatif.

3.6 Contraintes Réglementaires

Le déploiement d'IAS en France et dans l'Union Européenne s'inscrit dans un cadre réglementaire qu'il faut prendre en compte même si la plupart des contraintes jouent en notre faveur.

Sur le RGPD : IAS analyse du code source et non des données personnelles. Cela dit, le code audité peut contenir des données sensibles comme des clés API ou des identifiants. Le fait que l'IA tourne entièrement en local via Ollama garantit qu'aucune donnée ne sort de l'infrastructure du client. La conformité RGPD est donc assurée par conception.

Sur l'AI Act : IAS est un outil d'aide à la décision pour développeurs ce qui le classe dans la catégorie « risque limité ». Il ne prend aucune décision automatisée impactant des personnes et les rapports indiquent clairement quand les résultats proviennent de l'analyse IA.

Sur NIS 2 : en tant qu'outil de détection de vulnérabilités IAS contribue positivement à la conformité NIS 2 des entreprises qui l'utilisent. C'est davantage un argument commercial qu'une contrainte.

Sur la souveraineté numérique : la stack 100 % open-source combinée à l'exécution locale répond directement aux préoccupations croissantes de souveraineté des données. C'est un argument particulièrement fort auprès des administrations ainsi que des entreprises de secteurs stratégiques.

3.7 Positionnement Marché et Avantage Concurrentiel

IAS se positionne comme une alternative économique et agile aux solutions d'audit établies. Le tableau suivant permet de comparer rapidement :

Critère	CAST	SonarQube	IAS
Coût / 25 apps	~30 000 €	Gratuit (CE)	< 6 000 €
Analyse architecturale	Oui (propriétaire)	Non	Oui (open-source)
Détection CVE/CWE	Oui	Partielle	Oui (OWASP + IA)
Analyse IA du code	Non	Non	Oui (Mistral)
Temps de formation	Plusieurs semaines	Quelques jours	< 1 journée
Souveraineté données	Cloud éditeur	Auto-hébergeable	100 % local
Formats de sortie	Propriétaires	Dashboard web	JSON, HTML, Graphe

Le modèle économique repose sur l'intégration d'IAS dans les prestations de services outillés d'Inetum. Chaque audit réalisé avec l'outil renforce la proposition commerciale en montrant une capacité d'analyse rapide et approfondie ce qui fait la différence lors des appels d'offres. La durabilité du modèle est assurée par la nature récurrente des audits ainsi que par l'élargissement progressif du périmètre fonctionnel de l'outil.

3.8 Impacts Environnementaux et Éco-Conception

Plusieurs choix d'architecture contribuent à limiter l'empreinte environnementale de la solution. Le plus structurant est l'exécution locale de l'IA : un modèle Mistral 7B quantifié, exécuté via Ollama, consomme significativement moins qu'un appel à un LLM hébergé sur des clusters GPU distants. Le cahier des charges identifie d'ailleurs explicitement le modèle d'IA comme le composant à plus fort impact environnemental, ce qui justifie ce choix.

À cela s'ajoutent deux leviers complémentaires : l'architecture événementielle via RabbitMQ, qui ne sollicite les ressources de calcul que lorsqu'un job est effectivement à traiter, et la conteneurisation Docker, qui permet un dimensionnement précis et évite le surdimensionnement de serveurs tournant à vide. L'analyse incrémentale, prévue dans les évolutions futures, constituera un levier supplémentaire en ne ré-analysant que les fichiers modifiés entre deux scans.

3.9 Programme de Travail et Jalons

Le développement suit une approche itérative structurée autour de l'échelle TRL (Technology Readiness Level) :

Phase	TRL	Objectif	Livrables	Période
Phase 1	TRL 1-3	Recherche et faisabilité	CDC, état de l'art, choix techno	Nov 2025
Phase 2	TRL 4	POC fonctionnel	Pipeline 8 étapes, rapport HTML	Déc 2025 – Fév 2026
Phase 3	TRL 5-6	Validation terrain	Tests sur cas d’usages Inetum réels, en environnement UAT	Mars – Avr 2026
Phase 4	TRL 7-8	Pré-production	Multi-langage, visualisations	Mai – Juil 2026
Phase 5	TRL 9	Déploiement	Intégration services Inetum	Sep 2026+

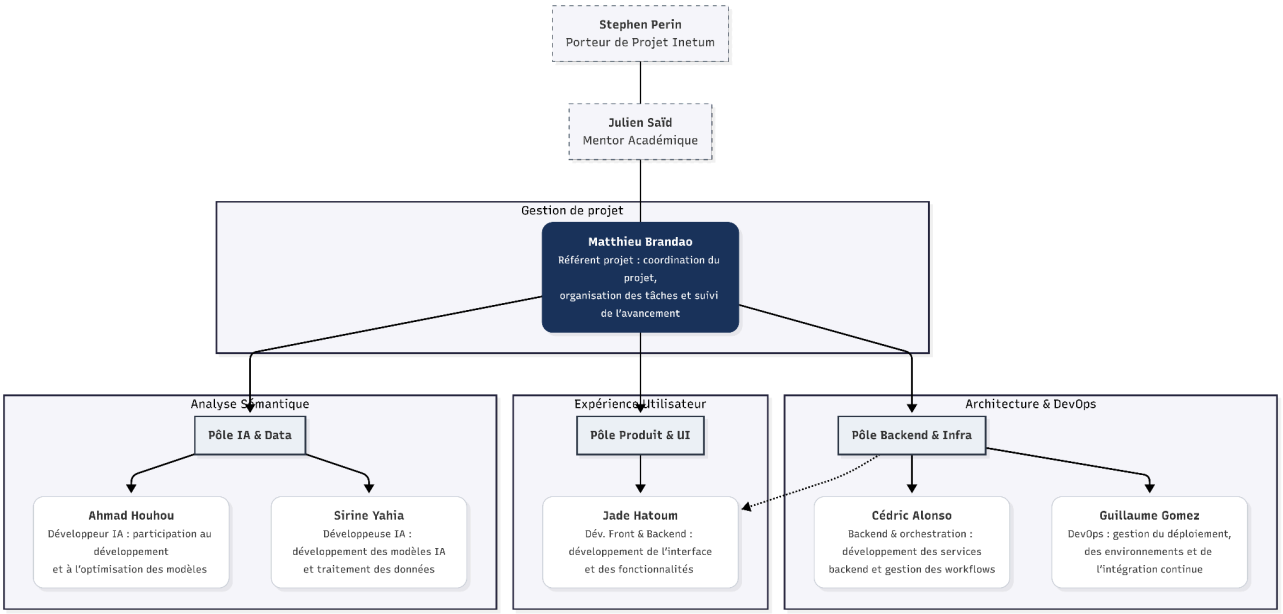
On en est aujourd'hui à la Phase 2 c'est-à-dire TRL 4. Le POC a validé les briques techniques fondamentales à savoir l'ingestion de code puis le scan de dépendances puis l'analyse IA puis le stockage en graphe et enfin la génération de rapports. Tout fonctionne de bout en bout.

Les prochaines étapes visent à élargir le support linguistique avec Java et PHP en priorité puisque le cahier des charges cible initialement l'écosystème Java. Il s'agira aussi de développer les visualisations architecturales interactives avec des outils comme Mermaid.js ou Structurizr et d'intégrer l'outil dans les pipelines CI/CD pour un usage en production.

5. Fonction et contribution des acteurs



Notre équipe souhaite prouver qu'une alternative open-source basée sur l'IA est réalisable. Chaque membre est en alternance dans un grand groupe (Société Générale, Dassault Systèmes, CACIB, Sephora, Colas, Memority) ce qui ancre notre travail dans la réalité du terrain.



6. Études de faisabilité technique réalisées

6.1 Environnement physique

Le POC d'Inetum App Scanner ne requiert aucun équipement physique spécifique ni configuration spatiale particulière. L'intégralité du système s'exécute sur des machines standards (postes de développement ou serveurs virtuels) sans contrainte géographique ou d'emplacement. Aucun périphérique dédié (capteurs, bornes, écrans spécifiques) n'est nécessaire.

Les tests peuvent être exécutés depuis n'importe quel environnement disposant d'un accès réseau local ou distant vers les conteneurs Docker du système. En contexte de démonstration ou de validation par le jury, un poste connecté à Internet avec un navigateur web moderne suffit pour accéder à l'interface utilisateur et consulter les rapports générés.

6.2 Environnement matériel

La configuration matérielle minimale requise pour exécuter le POC est la suivante :

Composant	Configuration minimale
Processeur	CPU multi-cœur x86_64 (Intel i5 / AMD Ryzen 5 ou équivalent)
Mémoire vive	16 Go RAM minimum (les conteneurs Neo4j, Ollama et ChromaDB sont gourmands en mémoire)
Stockage	30 Go d'espace disque libre (images Docker, modèles Ollama, cache NVD, données Neo4j)
GPU (recommandé)	GPU NVIDIA avec 6 Go VRAM pour accélérer l'inférence Mistral via Ollama. Sans GPU l'analyse fonctionne en mode CPU mais les temps de traitement augmentent significativement.
Réseau	Connexion Internet pour le téléchargement initial des images Docker, des modèles IA (Mistral, nomic-embed-text) et de la base NVD. Aucune connexion requise ensuite en fonctionnement.

Pour les tests de validation, un poste client disposant d'un navigateur web récent (Chrome, Firefox, Edge) est suffisant pour accéder au frontend React sur le port 5173 et à l'API sur le port 8080.

6.3 Environnement logiciel

L'ensemble de la stack logicielle externe nécessaire au fonctionnement du POC est orchestrée via Docker Compose. Aucune installation manuelle de dépendances n'est requise en dehors de Docker lui-même.

Logiciel	Version	Licence	Rôle
Docker / Docker Compose	24+	Apache 2.0	Conteneurisation et orchestration de l'infrastructure

Neo4j	5.26	GPL v3 (Community)	Base de données graphe pour les dépendances et vulnérabilités (File → Class → Method → Vulnerability)
ChromaDB	0.4.24	Apache 2.0	Mémoire vectorielle pour les embeddings du code source
RabbitMQ	3 (management)	MPL 2.0	Broker de messages pour l'orchestration asynchrone du pipeline (avec retry et DLQ)
MongoDB	7	SSPL	Persistance de l'état des jobs d'analyse
Azurite	Latest	MIT	Émulateur Azure Blob Storage pour le stockage des archives et rapports
Ollama	Latest	MIT	Serveur d'inférence IA locale pour Mistral et nomic-embed-text
OWASP Dep-Check	Latest	Apache 2.0	Scan des dépendances pour détection CVE/CVSS via la base NVD
.NET SDK	8.0	MIT	SDK pour la compilation de l'API et du Worker (uniquement en développement)
Node.js / npm	20+	MIT	Build du frontend React/TypeScript (uniquement en développement)

Le démarrage complet de l'environnement de test se fait via une seule commande : docker compose up. Les modèles Mistral et nomic-embed-text sont automatiquement téléchargés par le conteneur Ollama au premier lancement.

6.4 Données

Les données nécessaires à l'exécution de la suite de tests sont de deux natures :

Données d'entrée (projets à analyser)

Conformément à la contrainte méthodologique du CdC (cf. 4.7.2), seuls des projets open-source sont utilisés. Le POC cible actuellement l'écosystème C# / .NET. Les jeux de test comprennent des projets de tailles variées : un petit projet (quelques dizaines de fichiers .cs, moins de 5 000 lignes) pour les tests de validation nominaux, un projet moyen (plusieurs centaines de fichiers, de l'ordre de 50 000 lignes) pour les tests de performance, et un projet contenant des vulnérabilités connues (dépendances NuGet obsolètes référencées dans la NVD) pour valider la détection CVE. Les projets sont soumis sous forme d'archive ZIP ou via URL de dépôt Git.

Données de référence

La base NVD (National Vulnerability Database) est utilisée par OWASP Dependency-Check pour la détection des CVE. Elle est téléchargée automatiquement et mise en cache localement. Le modèle Mistral 7B (via Ollama) est utilisé pour l'analyse sémantique du code source. Le modèle nomic-embed-text génère les embeddings vectoriels indexés dans ChromaDB. Aucune donnée personnelle ou confidentielle n'est manipulée conformément aux exigences RGPD.

6.5 Analyse de marché et positionnement concurrentiel

Le POC s'inscrit dans un marché dominé par deux acteurs principaux : CAST (Imaging et Highlight) et SonarQube. Le tableau comparatif suivant positionne IAS par rapport à ces solutions :

Critère	CAST	SonarQube	IAS (POC)
Coût / 25 apps	~30 000 €	Gratuit (CE)	< 6 000 € (objectif -60 à -80%)
Analyse architecturale	Oui (propriétaire)	Non	Oui (open-source, graphe Neo4j)
Détection CVE/CWE	Oui	Partielle	Oui (OWASP + IA)
Analyse IA du code	Non	Non	Oui (Mistral local)
Souveraineté données	Cloud éditeur	Auto-hébergeable	100% local
Temps de formation	Plusieurs semaines	Quelques jours	< 1 journée

Le caractère innovant repose sur la combinaison inédite de trois approches : un scan de dépendances classique (OWASP), une analyse sémantique par LLM (Mistral/Ollama) et un fallback heuristique (regex). L'approche RAG (Retrieval-Augmented Generation) via ChromaDB pour l'audit applicatif est nouvelle dans le domaine. La modélisation en graphe (Neo4j) des relations File → Class → Method → Vulnerability offre une cartographie navigable, ce qui se distingue nettement des rapports PDF statiques des outils classiques.

6.6 Évaluation des risques techniques

Risque	Probabilité	Impact	Mitigation
Fenêtre de contexte LLM dépassée	Haute	Fichiers longs tronqués, analyse incomplète	Troncature à 6000 caractères par fichier dans le prompt + embeddings ChromaDB pour le contexte sémantique
Réponse IA non parsable	Moyenne	Perte de couverture d'analyse	Scanner heuristique regex en fallback automatique (RegexHeuristicScanner)
Performance sur gros projets	Haute	Temps d'analyse excessif	Architecture pipeline asynchrone (RabbitMQ), politique de retry (3 tentatives), future parallélisation
Indisponibilité Ollama	Faible	Pipeline bloqué aux étapes IA	Health checks, timeout configurable (15 min), DLQ pour les jobs en échec définitif
Faux positifs IA	Moyenne	Rapports pollués de fausses alertes	Seuil de risque minimum (score ≥ 4/10) pour filtrer les résultats non significatifs

6.7 Approche éco-conception

L'impact environnemental a été pris en compte dès les choix d'architecture. L'exécution locale de l'IA via Ollama (modèle Mistral 7B quantifié) évite les allers-retours vers des API cloud hébergées sur des clusters GPU distants et réduit considérablement l'empreinte carbone par rapport à un appel à un LLM cloud.

L'architecture événementielle via RabbitMQ permet de ne solliciter les ressources GPU que durant les étapes d'analyse IA et d'indexation vectorielle et non en permanence. La conteneurisation Docker permet un dimensionnement précis des ressources évitant le surdimensionnement permanent de serveurs. L'analyse incrémentale prévue dans les évolutions futures réduira significativement le volume de traitement récurrent.

6.8 Scalabilité et contraintes opérationnelles

L'architecture modulaire du POC (API, Worker, services de stockage) permet une mise à l'échelle progressive. Chaque composant est isolé dans son conteneur Docker et peut être répliqué indépendamment. Le Worker écoute RabbitMQ et peut être instancié plusieurs fois pour paralléliser le traitement des jobs. Le pipeline est configurable via appsettings.json ce qui permet d'ajouter ou retirer des étapes sans modification du code.
Les contraintes opérationnelles identifiées pour le passage à l'échelle sont les suivantes : la latence d'inférence Ollama sur CPU est le principal goulet d'étranglement (un GPU dédié est fortement recommandé en production), la base NVD nécessite un téléchargement initial d'environ 5 Go et la taille des projets analysables est limitée par l'espace disque disponible. Pour une mise en production, le choix de la région cloud et de son mix énergétique reste un levier d'optimisation important.

7. Suite de tests

Cette section organise les tests à exécuter selon les exigences fonctionnelles et non-fonctionnelles du CdC. Chaque exigence définit un ou plusieurs tests qui valident la fonctionnalité requise. Le jury dispose d'un espace OK/NOK pour noter la validation de chaque test.

7.1 Exigence 1 : Ingestion des données

(F1 - cf. CdC 4.5.1.1)

Objectif : Le système doit être capable d'ingérer du code source sous forme d'archive ZIP ou de dépôt Git, de le stocker dans le blob storage (Azurite) et de créer un job de traitement associé.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Upload ZIP valide	L'utilisateur soumet une archive .zip contenant un projet C# via l'interface React (drag & drop) ou via l'API POST /ias/jobs/zip.	Le système retourne un statut 202 Accepted avec un identifiant de job. Le job apparaît dans la liste avec le statut « Created ».	OK
Upload ZIP invalide (fichier non-zip)	L'utilisateur soumet un fichier .txt ou .pdf via le endpoint ZIP.	Le système retourne une erreur 400 Bad Request avec un message explicatif. Aucun job n'est créé.	OK
Ingestion via dépôt Git	L'utilisateur soumet une URL de dépôt Git public via POST /ias/jobs/git avec un JSON contenant repositoryUrl et branch.	Le système clone le dépôt, crée le job et retourne un statut 202 Accepted.	OK
Lancement de l'analyse (enqueue)	L'utilisateur lance l'analyse d'un job créé via POST /ias/jobs/{id}/enqueue.	Le job passe au statut « Queued » puis « Running ». Le message est publié dans la queue RabbitMQ.	OK
Rate limiting	L'utilisateur soumet plus de 5 requêtes par minute sur les endpoints de création.	Le système retourne un statut 429 Too Many Requests à partir de la 6e requête.	OK

7.2 Exigence 2 : Pipeline d'analyse complète

(F1.2, F2, F3 - cf. CdC 4.5.1.2 à 4.5.1.6)

Objectif : Le pipeline en 8 étapes doit s'exécuter de bout en bout (Download → Cleanup → Dependency Scan → Vector Index → AI Analysis → Audit Reporting → HTML Report → Upload) et produire un rapport complet.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Exécution complète du pipeline	Un projet C# valide est soumis et lancé. Le Worker consomme le message RabbitMQ et exécute les 8 étapes séquentiellement.	Le job passe par les états Running → Completed. Les fichiers dependency-vulnerabilities.json, code-source-audit.json et report.html sont générés dans le répertoire de sortie.	OK

Nettoyage du graphe (Cleanup)	Un job est lancé après une analyse précédente.	Le graphe Neo4j est vidé de ses nœuds non protégés avant la nouvelle analyse. Les données de l'analyse précédente ne polluent pas les résultats.	OK
Indexation vectorielle	Le Worker traite un projet contenant 20 fichiers .cs.	Chaque fichier est indexé dans ChromaDB via nomic-embed-text. Le graphe Neo4j contient les nœuds File → Class → Method correspondants.	OK
Analyse IA (Mistral)	Le Worker analyse un fichier contenant une injection SQL volontaire (concaténation de chaînes dans une requête SQL).	Mistral détecte la vulnérabilité et retourne un JSON valide avec issueFound: true et un riskScore ≥ 4. La vulnérabilité est enregistrée dans Neo4j.	OK
Fallback heuristique	Le Worker analyse un fichier pour lequel Mistral retourne une réponse non parsable (JSON invalide).	Le scanner heuristique regex (RegexHeuristicScanner) prend le relais et détecte les patterns suspects. Une vulnérabilité de type « Heuristic Fallback » est créée.	OK

7.3 Exigence 3 : Détection des vulnérabilités

(cf. CdC 4.2.2 et 4.8.4)
Objectif : Le système doit identifier les CVE des dépendances via OWASP Dependency-Check et les vulnérabilités du code source via Mistral, puis les consolider dans les rapports.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Détection CVE sur dépendances	Un projet contenant des dépendances NuGet avec des CVE connues est soumis.	Le fichier dependency-vulnerabilities.json liste les CVE identifiées avec leur CveId, Dependency, Severity et CvssScore (v2 ou v3).	OK
Détection vulnérabilités code source	Un projet contenant du code vulnérable (secrets en dur, injections SQL) est soumis.	Le fichier code-source-audit.json contient les vulnérabilités avec ClassName, VulnerabilityType, RiskScore, DetailedAnalysis et Remediation.	OK
Consolidation des résultats dans le graphe	Après analyse complète, interrogation du graphe Neo4j.	Les nœuds Vulnerability sont liés aux nœuds Class via la relation HAS_VULNERABILITY. Les propriétés vulnerabilityType, detailedAnalysis, remediation et riskScore sont présentes.	OK

Extraits de code dans le rapport d'audit	Analyse d'un projet dont les fichiers sources sont accessibles.	Chaque entrée du rapport code-source-audit.json inclut les AffectedFiles avec le chemin du fichier, les lignes de début/fin et un extrait de code contextualisé.	OK

7.4 Exigence 4 : Génération de rapports

(cf. CdC 4.2.4 et 4.8.1)
Objectif : Le système doit produire un rapport HTML de qualité professionnelle combinant les vulnérabilités de dépendances et l'audit du code source, consultable via le frontend ou l'API.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Génération du rapport HTML	Après complétion du pipeline, le rapport est généré via le moteur de template Scriban.	Le fichier report.html est créé, contient l'identifiant du job, la date d'analyse, les vulnérabilités de dépendances avec badges de sévérité (CRITICAL, HIGH, MEDIUM, LOW) et les résultats d'audit de code.	OK
Consultation du rapport via API	L'utilisateur appelle GET /ias/jobs/{id}/report pour un job terminé.	Le système retourne un JSON contenant le HTML du rapport. Le statut est 200 OK.	OK
Consultation du rapport via frontend	L'utilisateur clique sur « View Report » dans le drawer de détail d'un job complété.	Un nouvel onglet s'ouvre avec le rapport HTML rendu correctement. Les tableaux, badges et code blocks sont lisibles.	OK
Rapport pour job non terminé	L'utilisateur appelle GET /ias/jobs/{id}/report pour un job en cours ou échoué.	Le système retourne un statut 409 Conflict avec un message explicatif.	NOK

7.5 Exigence 5 : Résilience et gestion des erreurs

(cf. CdC 4.3.4)
Objectif : Le pipeline doit gérer les erreurs de manière robuste avec un mécanisme de retry et de Dead Letter Queue pour éviter les pertes silencieuses.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Retry automatique après échec	Un job échoue lors de l'étape AI Analysis (par exemple Ollama temporairement indisponible).	Le job est republié dans la queue Retry avec un délai de 5 secondes. Le statut passe à « Retrying ». Après le délai le job est reconsommé et le pipeline reprend.	NOK

Échec définitif (3 tentatives)	Un job échoue systématiquement après 3 tentatives.	Le job passe au statut « Failed » avec le message d'erreur. Le message est publié dans la Dead Letter Queue. Le frontend annonce l'échec.	OK
Annulation propre à l'arrêt du Worker	Le Worker est arrêté (SIGTERM) pendant l'exécution d'un job.	Le job en cours est marqué « Canceled » en base MongoDB. Le message est acquitté dans RabbitMQ. Le frontend affiche la date d'annulation.	OK
Reprise après échec partiel	Un job échoue à l'étape 5 (AI Analysis) puis est réessayé.	Grâce au suivi du CurrentStepIndex, le pipeline reprend à l'étape 5 sans réexécuter les étapes 1 à 4 déjà complétées.	NOK

7.6 Exigence 6 : Performance de l'analyse

(cf. CdC 4.3.3 et 4.8.2)

Objectif : L'analyse doit se terminer dans un temps raisonnable : moins de 30 minutes pour un petit projet, compatible avec une fenêtre nocturne pour les grands projets.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Temps d'analyse petit projet	Soumission d'un projet de moins de 5 000 lignes de code (une vingtaine de fichiers .cs).	L'analyse complète (8 étapes) se termine en moins de 30 minutes.	OK
Temps d'analyse projet moyen	Soumission d'un projet d'environ 50 000 lignes de code.	L'analyse se termine en moins de 4 heures. Le suivi de progression dans le frontend est fonctionnel.	OK
Timeout du scan de dépendances	Soumission d'un projet avec un très grand nombre de dépendances.	Si OWASP Dependency-Check dépasse le timeout configuré (15 minutes), l'étape échoue proprement et le mécanisme de retry prend le relais.	OK

7.7 Exigence 7 : Utilisabilité

(cf. CdC 4.3.2)
Objectif : L'interface utilisateur doit être intuitive et ne nécessiter aucune formation technique approfondie pour soumettre un projet et consulter les résultats.

Description	Scénario d'exécution	Résultats attendus	OK/NOK
Upload et lancement depuis le frontend	Un utilisateur non technique ouvre l'interface React, uploade un ZIP et lance l'analyse.	L'ensemble du parcours (upload → enqueue → suivi → rapport) est réalisable en moins de 5 clics. La progression est visible en temps réel.	OK

Suivi des jobs	L'utilisateur consulte la page Jobs listant tous les jobs soumis.	La liste affiche pour chaque job : son identifiant, son statut (Created, Queued, Running, Completed, Failed, Canceled) et les horodatages clés. Un clic ouvre le drawer de détail avec la timeline complète.	OK
Health check de l'API	Appel à GET /health.	Le système retourne un statut 200 confirmant que l'API est opérationnelle.	OK

7.8 Validation par les parties prenantes

7.8.1 Validation fonctionnelle

Les tests fonctionnels couvrent l'ensemble des capacités clés du POC : ingestion de données, exécution du pipeline, détection des vulnérabilités et génération de rapports. Ces tests ont été conçus à partir des cas d'usage définis dans le CdC et validés en interne par l'équipe. La validation finale sera effectuée par le client Inetum (Stephen Perin) et le mentor académique (Julien Saïd) lors de la démonstration du POC.

7.8.2 Validation de performance

Les tests de performance mesurent les temps d'exécution sur des jeux de données représentatifs. Ces mesures seront comparées aux objectifs du CdC (moins de 30 minutes pour un petit projet, fenêtre nocturne pour les grands projets). Les tests de résilience valident le comportement du système en conditions dégradées.

7.8.3 Validation utilisateurs et retours externes

La validation avec les utilisateurs finaux (architectes, développeurs, DevSec) sera réalisée lors de la phase UAT prévue en Phase 3 (TRL 5-6, mars-avril 2026). Pour le POC actuel (TRL 4), la validation repose sur la démonstration aux parties prenantes Inetum et au jury académique, avec collecte de retours structurés sur la lisibilité des rapports, la clarté de l'interface et la pertinence des vulnérabilités détectées. Ces retours alimentent les itérations suivantes conformément à la feuille de route définie dans le CdC (cf. section 5).

8. Résultats, Analyse et Interprétation

8.1 Protocoles de collecte des données

Les résultats du POC sont collectés via trois canaux complémentaires. Le pipeline génère automatiquement deux fichiers JSON structurés (dependency-vulnerabilities.json et code-source-audit.json) ainsi qu'un rapport HTML consolidé. Le graphe Neo4j est interrogeable via des requêtes Cypher pour extraire les relations File → Class → Vulnerability. Les logs applicatifs (.NET ILogger) tracent l'exécution de chaque étape du pipeline avec les temps de traitement et les erreurs éventuelles.

Pour les tests de performance les temps d'exécution sont mesurés via les horodatages enregistrés dans MongoDB (createdAt, enqueuedAt, startedAt, completedAt) ce qui fournit une mesure précise du temps global et de chaque phase.

8.2 Présentation des résultats du POC

Le POC a validé les briques techniques fondamentales du système. Le pipeline en 8 étapes fonctionne de bout en bout sur des projets C# réels. Voici les résultats observés sur les principaux axes :

8.2.1 Ingestion et structuration

L'ingestion par ZIP et par dépôt Git fonctionne correctement. Le système extrait les fichiers, les stocke dans Azurite et déclenche le pipeline via RabbitMQ. La structuration dans le graphe Neo4j (File → Class → Method) est opérationnelle et les contraintes d'unicité sur les nœuds Class et File sont respectées.

8.2.2 Détection des vulnérabilités

La double détection fonctionne comme prévu. OWASP Dependency-Check identifie les CVE connues dans les dépendances NuGet avec les scores CVSS v2/v3. Mistral analyse le code source fichier par fichier et détecte les failles structurelles (injections SQL, XSS, secrets en dur). Lorsque l'IA retourne une réponse non parsable le scanner heuristique regex prend le relais automatiquement garantissant une couverture minimale.

8.2.3 Génération des rapports

Les rapports JSON et HTML sont générés automatiquement via le moteur de template Scriban. Le rapport HTML inclut les badges de sévérité colorés (CRITICAL, HIGH, MEDIUM, LOW), les extraits de code contextualisés et les recommandations de remédiation. La consultation via le frontend ou l'API est fonctionnelle.

8.2.4 Résilience du pipeline

Le mécanisme de réessai automatique (3 tentatives, délai de 5 secondes entre chaque) a été validé : les jobs en échec sont automatiquement re-tentés puis, en cas d'épuisement des tentatives, marqués comme échoués et redirigés vers la Dead Letter Queue (DLQ) pour traçabilité. L'arrêt gracieux du Worker via CancellationToken a également été vérifié, garantissant qu'aucun job en cours n'est interrompu brutalement.

8.3 Méthodes d'analyse

8.3.1 Protocoles de validation des résultats

L'analyse des résultats repose sur trois approches complémentaires :

1. La vérification manuelle des rapports générés permet de confirmer que les vulnérabilités détectées correspondent bien à des failles réelles (pas de faux positifs critiques).
2. La comparaison avec des résultats connus (projets open-source dont les CVE sont documentées) valide le taux de détection de la couche OWASP.

3. Enfin le test unitaire existant (TestGenerateHtmlReportStep) valide la génération du rapport HTML par rapport à un jeu de données de référence.

8.3.2 Rigueur du cycle de développement (QA & DevOps)

La fiabilité du code source d'IAS est assurée par une chaîne de livraison logicielle stricte :

- **Stratégie de Branching :** Aucun changement n'est poussé directement sur la branche principale. Nous utilisons des **Pull Requests (PR)** systématiques depuis une branche development.
- **Peer-to-Peer Code Review :** Chaque PR est soumise à une revue de code croisée. Ce processus garantit que les algorithmes d'analyse (notamment les Regex du fallback) sont validés par au moins deux ingénieurs avant intégration.
- **Pipeline CI/CD GitLab :** Nous avons automatisé l'intégration continue via GitLab CI. À chaque commit, le pipeline exécute les tests et build les images Docker. Cela garantit que l'environnement de test est toujours conforme à l'environnement de production.

8.4 Interprétation et discussion

Les résultats du POC démontrent la faisabilité technique de l'approche proposée. La combinaison du scan de dépendances classique (OWASP) avec l'analyse sémantique par LLM (Mistral) et le fallback heuristique offre une couverture de détection plus large que chaque approche prise isolément.

Cependant plusieurs limites sont identifiées. La qualité de l'analyse IA dépend fortement du modèle utilisé : Mistral 7B est un bon compromis coût/performance mais un modèle plus grand améliorerait la précision. La troncature des fichiers à 6000 caractères peut entraîner des angles morts sur des classes volumineuses. Le seuil de risque minimum fixé à 4/10 filtre efficacement les faux positifs mais peut aussi masquer des vulnérabilités subtiles à faible score.

L'absence actuelle de visualisations architecturales interactives (graphes de dépendances navigables) constitue la principale limitation du POC par rapport à la vision complète du CdC. Cette fonctionnalité est prévue pour la Phase 3 avec l'intégration d'outils comme Mermaid.js ou Structurizr.

9. Conclusions et Feuille de route

9.1 Conclusions

Le POC d'Inetum App Scanner démontre la faisabilité technique d'une alternative crédible et économique aux solutions d'audit applicatif dominantes sur le marché. Les objectifs fondamentaux du POC sont atteints : l'ingestion de code source fonctionne via ZIP et Git, la structuration en graphe (Neo4j) est opérationnelle, la double détection des vulnérabilités (OWASP + IA) est fonctionnelle et la génération automatique de rapports HTML professionnels est validée.

La stack 100% open-source combinée à l'exécution locale de l'IA répond directement aux enjeux de souveraineté numérique et de conformité RGPD identifiés dans le CdC. L'architecture modulaire (API + Worker + services) et l'orchestration événementielle (RabbitMQ) garantissent la maintenabilité et l'évolutivité du système conformément aux exigences non-fonctionnelles.

Le principal livrable technique est un pipeline opérationnel en 8 étapes avec gestion robuste des erreurs (retry + DLQ), validé de bout en bout sur des projets C# réels. Ce résultat correspond au TRL 4 (validation en laboratoire) défini dans la feuille de route.

9.2 Projection et feuille de route

Les évolutions prévues s'articulent autour de quatre phases définies dans le CdC :

Phase	TRL	Objectif	Livrables clés
Phase 3	TRL 5-6	Validation terrain : tests sur projets réels et UAT	Support multi-langage (Java, PHP), visualisations interactives (Mermaid.js/Structurizr), intégration CI/CD
Phase 4	TRL 7-8	Pré-production : robustesse et couverture complète	Analyse d'obsolescence, score de maintenabilité, propositions de modernisation architecturale, analyse incrémentale
Phase 5	TRL 9	Déploiement : intégration dans les services Inetum	Intégration comme « service outillé » dans les offres d'audit Inetum, documentation utilisateur, support multi-tenant

Les priorités immédiates pour la Phase 3 sont l'élargissement du support linguistique à Java (cible initiale du CdC) et le développement des visualisations architecturales interactives qui représentent le besoin fonctionnel principal non encore couvert par le POC (cf. CdC 4.2.1). La parallélisation de l'analyse et l'analyse incrémentale viendront dans un second temps pour répondre aux exigences de performance sur des portefeuilles applicatifs volumineux.

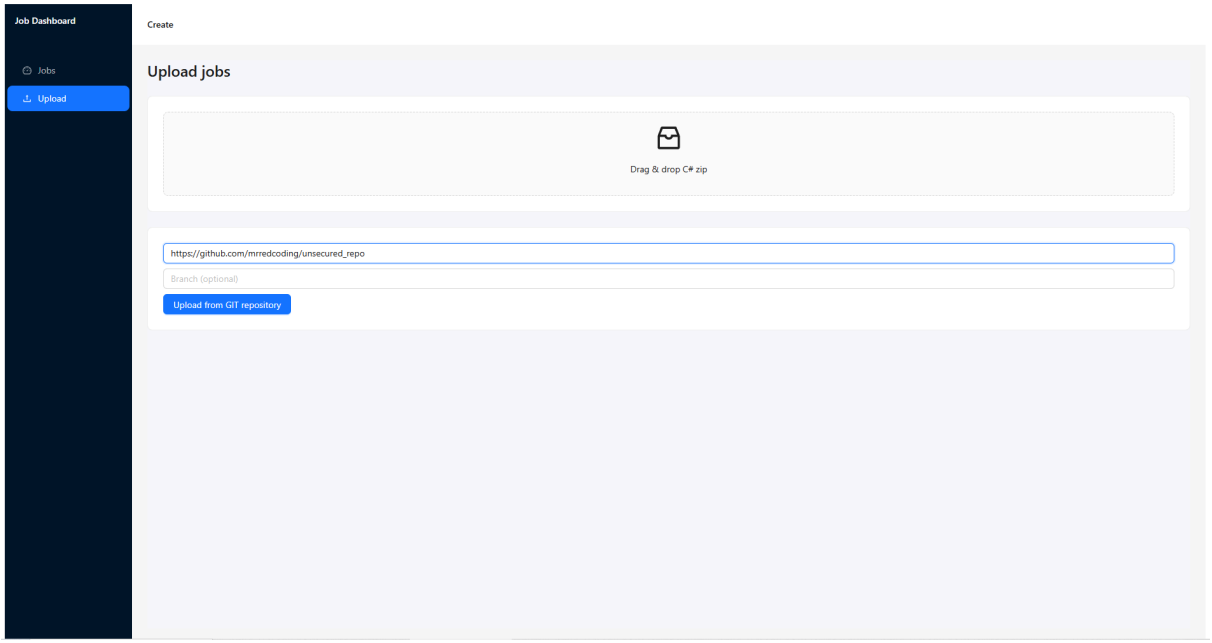
Annexe

Use Case : Utilisation fonctionnelle d'Inetum App Scanner

Cette annexe présente un exemple concret d'utilisation d'Inetum App Scanner du point de vue utilisateur. Elle décrit les principales interactions avec l'application et les différents états d'un job d'analyse, sans revenir sur les détails techniques internes déjà développés dans le rapport.

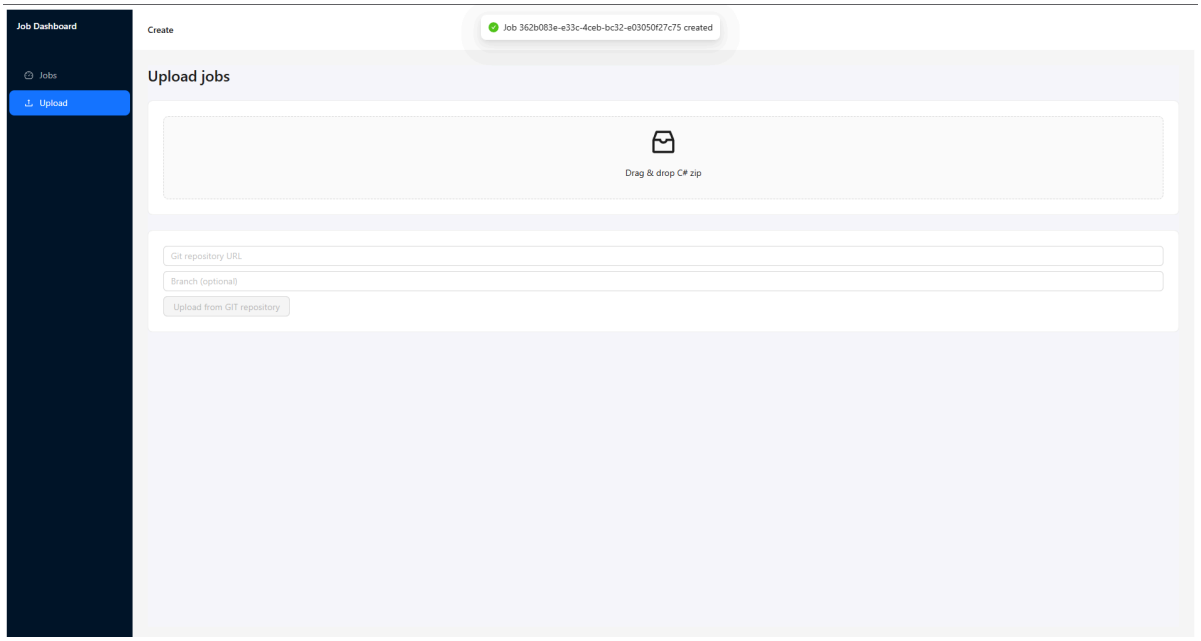
UC-01 – Soumission d'un projet

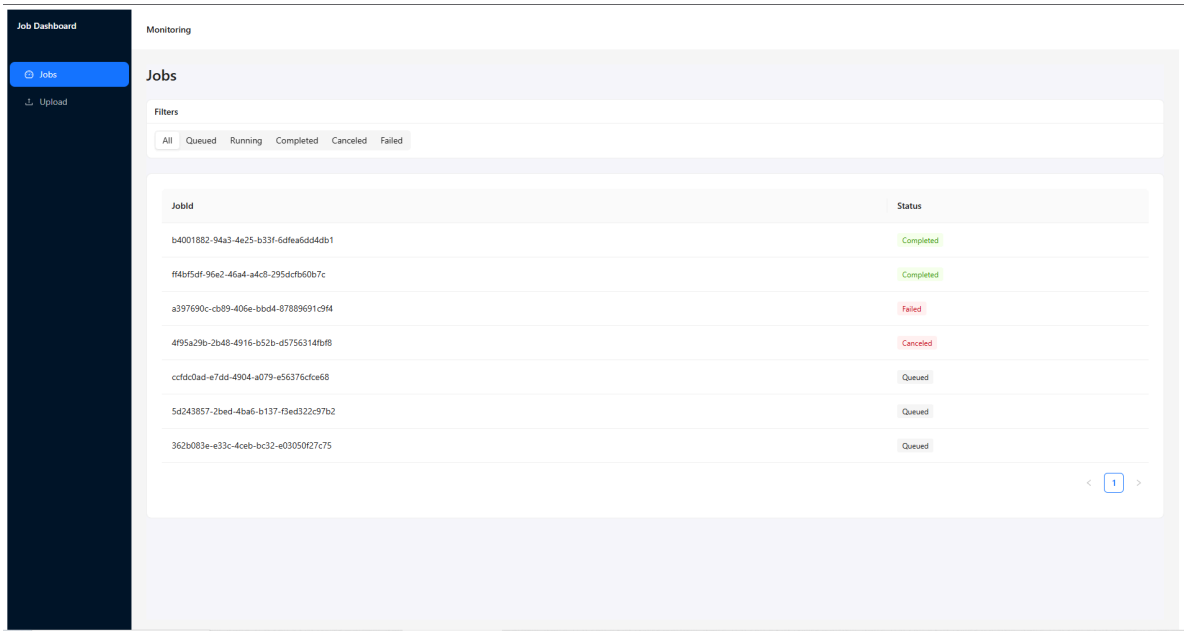
L'utilisateur accède à l'interface d'Inetum App Scanner et soumet un projet à analyser, soit sous la forme d'une archive ZIP, soit via l'URL d'un dépôt Git. Après validation, le système crée un job d'analyse et l'enregistre avec le statut *Created*. Le job apparaît immédiatement dans la liste des analyses disponibles.



UC-02 – Mise en file d'attente du job

L'utilisateur déclenche l'exécution du job. Celui-ci est placé en file d'attente et passe au statut *Queued*, indiquant qu'il est prêt à être traité mais en attente de ressources disponibles.



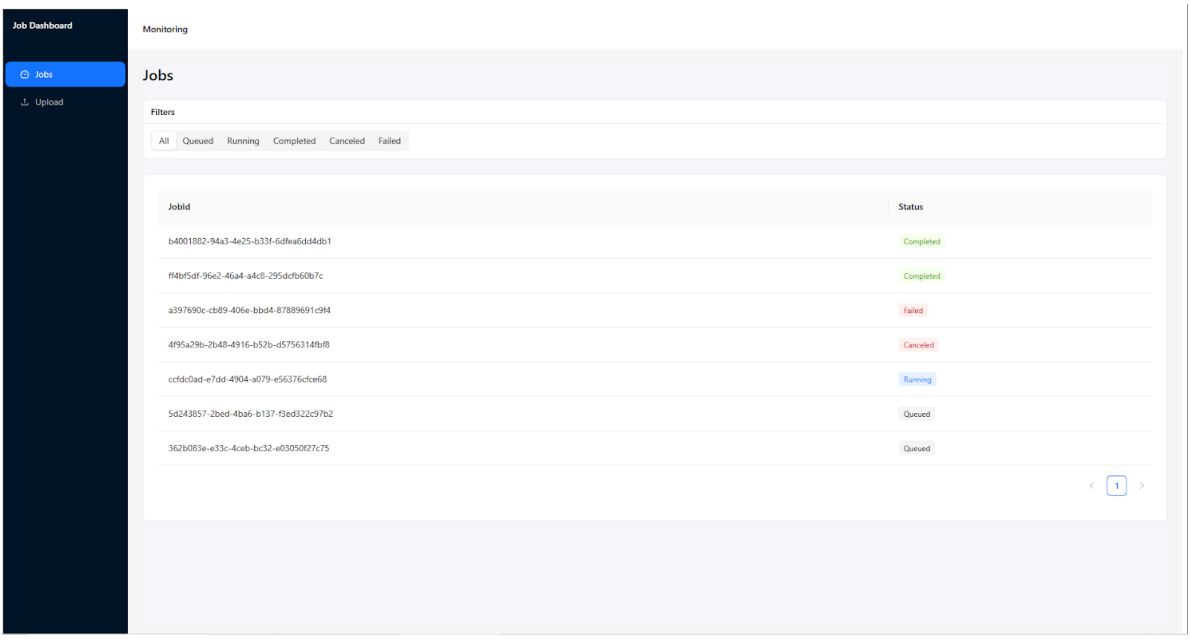


UC-03 – Démarrage automatique du traitement

Dès qu'un worker devient disponible, le job précédemment en file d'attente est automatiquement pris en charge par le système. Son statut passe alors de *Queued* à *Running*, signalant le début effectif de l'analyse sans action supplémentaire de l'utilisateur.

UC-04 – Suivi de l'exécution

Pendant l'analyse, l'utilisateur peut suivre en temps réel l'évolution du job depuis l'interface. Les changements de statut et les informations de progression sont mis à jour automatiquement, offrant une visibilité continue sur l'état du traitement.



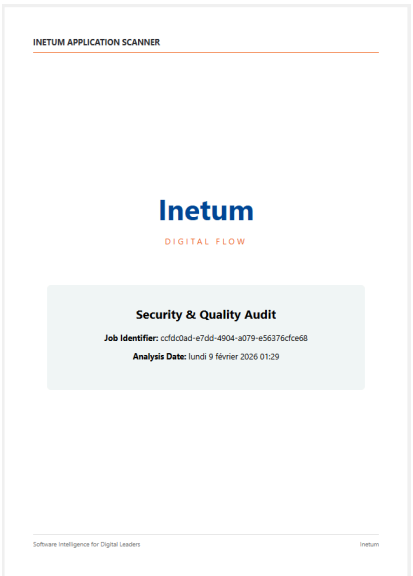
UC-05 – Gestion des erreurs

En cas d'erreur durant l'exécution, le système applique automatiquement sa politique de reprise. Si l'échec persiste après plusieurs tentatives, le job est marqué *Failed* et l'utilisateur est informé de l'état final de l'analyse.

UC-06 – Consultation des résultats

Lorsque l'analyse se termine avec succès, le job passe au statut *Completed*. L'utilisateur peut alors ouvrir le rapport généré au format HTML depuis l'interface, lequel présente de manière synthétique les vulnérabilités détectées dans les dépendances et le code source.




INETUM APPLICATION SCANNER

Inetum

DIGITAL FLOW

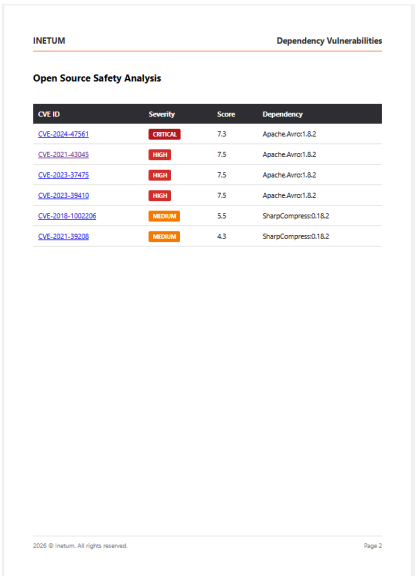
Security & Quality Audit

Job Identifier: ccfdc0ad-e7dd-4904-a079-e56376cfce68

Analysis Date: lundi 9 février 2026 01:29

Software Intelligence for Digital Leaders

Inetum


INETUM

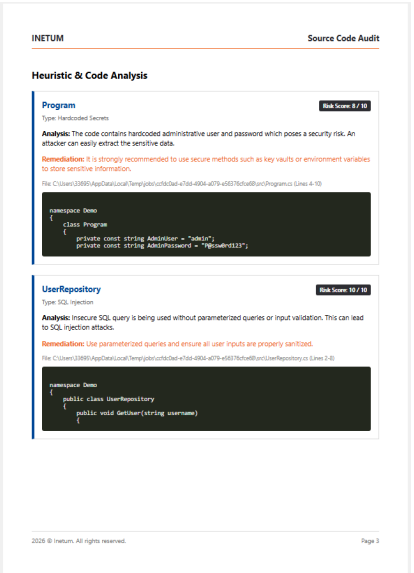
Dependency Vulnerabilities

Open Source Safety Analysis

CVE ID	Severity	Score	Dependency
CVE-2024-47561	CRITICAL	7.3	Apache.Avro:1.8.2
CVE-2021-43045	HIGH	7.5	Apache.Avro:1.8.2
CVE-2023-37475	HIGH	7.5	Apache.Avro:1.8.2
CVE-2023-39410	HIGH	7.5	Apache.Avro:1.8.2
CVE-2018-1002206	MEDIUM	5.5	SharpCompress:0.18.2
CVE-2021-39208	MEDIUM	4.3	SharpCompress:0.18.2

2026 © Inetum. All rights reserved.

Page 2


INETUM

Source Code Audit

Heuristic & Code Analysis

Program

Risk Score: 8 / 10

Type: Hardcoded Secrets

Analysis: The code contains hardcoded administrative user and password which poses a security risk. An attacker can easily extract the sensitive data.

Remediation: It is strongly recommended to use secure methods such as key vaults or environment variables to store sensitive information.

File: C:\Users\33695\AppData\Local\Temp\jobs\ccfdc0ad-e7dd-4904-a079-e56376cfce68\src\Program.cs (Lines: 4-10)

```
namespace Demo
{
    class Program
    {
        private const string AdminUser = "admin";
        private const string AdminPassword = "P@sswOrd123";
```

UserRepository

Risk Score: 10 / 10

Type: SQL Injection

Analysis: Insecure SQL query is being used without parameterized queries or input validation. This can lead to SQL injection attacks.

Remediation: Use parameterized queries and ensure all user inputs are properly sanitized.

File: C:\Users\33695\AppData\Local\Temp\jobs\ccfdc0ad-e7dd-4904-a079-e56376cfce68\src\UserRepository.cs (Lines: 2-8)

```
namespace Demo
{
    public class UserRepository
    {
        public void GetUser(string username)
        {
```

2026 © Inetum. All rights reserved.

Page 3